Integración por Monte Carlo en memoria compartida con OpenMP

Parte IV — Informe sobre el proceso de solución

Introducción a la computación paralela · Universidad Industrial de Santander · Docente: Sergio Augusto Gélvez Cortés · Repositorio: `DaGuishe/ExamenOpenMP` · Integrantes: [nombres y códigos]

1. Decisiones de diseño

Estructura. `montecarlo.c` conserva el ciclo del programa del docente (mismo `rand_r`, misma división entre `RAND_MAX` y misma condición $x^2 + y^2 \le 1$), pero recorre las N muestras en **bloques de 65 536**. Cada bloque llama a `muestrear_bloque`, que cuenta los aciertos de su tramo con un estado de generador **local**. `montecarlo_secuencial.c` es el programa del docente con validación de argumentos y medición de tiempo; `referencia/montecarlo_docente.c` es la copia literal del enunciado, para comparar.

Cláusulas OpenMP. `#pragma omp parallel for schedule(static) reduction(+:dentro)` sobre el ciclo de bloques. `reduction` da a cada hilo una copia privada de `dentro` y las suma al final; `static` asigna a cada hilo un rango contiguo, sin costo en tiempo de ejecución. p se fija con `omp_set_num_threads` y el programa informa los hilos realmente usados. No usamos `atomic` ni `critical`.

Semillas. Al revisar `rand_r` de glibc encontramos que es un LCG de 32 bits ($s \leftarrow 1103515245\,s + 12345 \bmod 2^{32}$, 3 pasos por llamada) con período completo 2^{32}. La semilla es entonces una *posición* en un único ciclo. En vez de dar a cada hilo "otra semilla", el bloque que empieza en la muestra i_0 **salta** $6i_0$ pasos desde la semilla base del docente (salto afín en $O(\log k)$). Así: (i) cada hilo tiene su estado privado; (ii) los tramos de los hilos son disjuntos por construcción, y con `static` el hilo t empieza en la muestra $\approx tN/p$; (iii) cada muestra recibe siempre los mismos números, y el resultado es **idéntico** para todo p e idéntico al del docente. Límite: $N \le 715\,827\,882$, para que $6N \le 2^{32}$. El programa verifica al iniciar que su `rand_r` es ese LCG.

2. Problemas encontrados y cómo se diagnosticaron

1. **El primer diseño no usaba `rand_r`.** El repositorio partió de una versión con un generador SplitMix64 indexado por muestra: era reproducible, pero no usaba el generador ni la secuencia del docente. Se rediseñó con el salto del LCG y la versión anterior quedó en `src/variantes/`.
2. **"Semilla distinta" no basta.** Programamos `solapamiento.c` para el esquema ingenuo $s_t = s + tN/p$. Con nuestra semilla, $N = 10^8$ y $p = 8$, el hilo 1 alcanza la semilla del hilo 0 tras 43 927 968 pasos, así que los dos generan **los mismos 5 178 672 puntos** (5,2 % de N). Con el salto no hay solapamiento.
3. **Tiempos ruidosos.** Las primeras mediciones en la máquina virtual (con la variante SplitMix64) dieron $p = 8$ más rápido que $p = 4$ y $S(4)$ entre 2,9 y 3,96 de una serie a otra. Por eso el protocolo final agrega una ejecución de calentamiento, intercala las configuraciones rotando el orden, hace 7 repeticiones y usa la mediana.
4. **Condiciones de carrera.** No aparecieron en la solución (la reducción y los estados locales las evitan). Para verificar que sabríamos detectarlas, `defectos.c` las provoca a propósito con $N = 10^7$ y $p = 4$. Con un **estado compartido**, `dentro` cambia en cada ejecución (7 854 178–7 858 533) y el programa es de 2 a 18 veces más lento. Con la **misma semilla** en todos los hilos, el resultado es siempre 7 858 232, que corresponde a $4\times$ el resultado de $N/4$ puntos. **Sin `reduction`** se pierde el 64–72 % de los aciertos ($\hat{\pi} \approx 0,9$–$1,1$).
5. **Errores menores en el desarrollo:** una advertencia de `-Wconversion` (conversión implícita `long long` → `double`), una fórmula de Gustafson que mezclaba f de Amdahl con α y un error estándar mal calculado. Se corrigieron antes de publicar (ver bitácora).

3. Validación

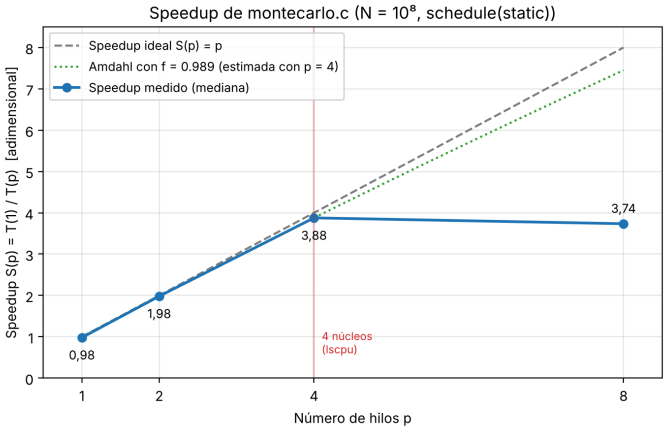
- **Estimación de π:** con $N = 10^8$, $\hat{\pi} = 3,14163332$ y error $4,07 \cdot 10^{-5}$, es decir $0,25\sigma$ con $\sigma = 4\sqrt{q(1-q)/N} = 1,64 \cdot 10^{-4}$. Las pruebas exigen $|\hat{\pi} - \pi| < 5\sigma$ con $N = 10^6, 10^7$ y 10^8.
- **Reproducibilidad con distinto p:** `dentro` es idéntico (no solo parecido) al del secuencial y al del programa del docente para $p = 1, 2, 3, 4, 8, 16$, con `static` y `dynamic` y tres semillas. Las 35 ejecuciones de la tabla dieron 78 540 833 aciertos.

- **Automatización:** `make test` ejecuta 45 comprobaciones: compilación sin advertencias (incluido `-Wpedantic -Wconversion`), 19 argumentos inválidos, N pequeño, pruebas unitarias del salto, igualdad exacta y un control que confirma que la versión sin `reduction` sí se detecta. `tests/test_resultados.py` recalcula medianas, speedup y eficiencia desde los CSV.

4. Resultados

Equipo: Intel Xeon 2,1 GHz, 4 núcleos (VM KVM), Ubuntu 24.04, gcc 13.3 (`resultados/equipo.txt`); $N = 10^8$, 7 ejecuciones por fila.

p	**Mediana $T(p)$ [s]**	$S(p)$	$E(p)$	**Karp-Flatt e**
1 (secuencial)	0,8735	1,000	1,000	—
1 (OpenMP)	0,8908	0,981	0,981	—
2	0,4408	1,982	0,991	0,0092
4	0,2253	3,877	0,969	0,0105
8	0,2338	3,735	0,467	0,1631



$T(1)$ es la mediana del programa secuencial real (7 ejecuciones, $N = 10^8$). La mediana con $p = 4$ resultó en 0,2253 s porque el problema está limitado por cómputo: no lee memoria, cada hilo trabaja en registros y la única sincronización es la reducción final. Por eso la eficiencia es 0,97. El 3 % restante no es código secuencial (lo estrictamente serial cuesta microsegundos): corresponde a la sobrecarga de la versión OpenMP (+2,0 % con $p = 1$) y a la variación del equipo. De $S(4)$ resulta $f = (1 - 1/3,877)/(1 - 1/4) = 0,9895$; Amdahl predice entonces $S(8) = 7,45$, pero medimos 3,74. La máquina tiene **4 núcleos**: 8 hilos se turnan en ellos y $T(8) \approx T(4)$, lo que explica el salto de e de 0,01 a 0,16. Con $f = 0,9895$, una máquina de 64 núcleos daría $S(64) \approx 38,5$ con N fijo, pero hasta 61 si N crece con p (Gustafson). Aumentar p no mejora la precisión, que solo depende de N ($\sigma \propto 1/\sqrt{N}$). En II.5, `static` y `dynamic` no difieren de forma medible con $p = 4$ (0,238 frente a 0,233 s, dentro del ruido).

5. Reflexión

El paralelismo de Monte Carlo es “trivial” solo si el generador se trata como parte del diseño: la parte delicada no fue la directiva `parallel for`, sino garantizar que cada muestra tuviera una identidad estable en el flujo aleatorio. Pensar las semillas como posiciones de un ciclo resolvió a la vez la independencia, el no solapamiento y la reproducibilidad, y permitió validar con igualdad exacta en lugar de “resultados parecidos”. También quedó claro que medir bien cuesta tanto como programar: en una máquina virtual el ruido supera las diferencias que se quieren estudiar, así que la mediana, el intercalado y el registro de cada ejecución son necesarios. **En un próximo trabajo** usaríamos desde el principio un generador con subflujos formales y período mayor (por ejemplo, PCG o xoshiro con `jump`), mediríamos en una máquina dedicada con más núcleos y frecuencia fija, y escribiríamos las pruebas de reproducibilidad antes que el código paralelo. [Espacio para que cada integrante añada, con sus palabras, lo que aprendió.]

Anexo — Bitácora de uso de IA

Declaración exigida por la instrucción general 3 y por los entregables del examen. Se registra cada consulta hecha a una herramienta de IA generativa, qué aportó, qué decidió la herramienta, cómo se verificó y qué queda por evaluar. **Las columnas "Evaluación del estudiante" están vacías a propósito:** solo los integrantes pueden decir qué aceptan, después de revisarlo y de ser capaces de explicarlo en la sustentación.

Campo	Valor
Herramienta	Claude Code (Anthropic), agente de programación en una sesión en la nube con acceso al repositorio. **Fue la única herramienta de IA usada** en el examen (declarado por el equipo).
Fecha	27 de octubre de 2026 (una sola sesión)
Usuario	Integrante dueño de `DaGuishe/ExamenOpenMP`
Alcance	La herramienta leyó el enunciado (PDF), escribió código, ejecutó compilaciones, pruebas y mediciones, redactó documentos e hizo commits

1. Consultas realizadas

#	Consulta del estudiante (resumen)	Propósito	Lo que aportó la herramienta
1	"Combina lo que hay en mi repo con lo que hay en el repositorio de mi compañero, termina lo que hace falta del examen"	Unificar los repositorios del equipo	Merge del historial de `Anderzz0n/Montecarlo_con_Openmp` (versión SplitMix64 del compañero), scripts de benchmark y una primera tabla de speedup. Sin el enunciado, supuso qué faltaba.
2	"Haz lo que hace falta en el repositorio de mi compañero"	Aplicar lo mismo en el otro repositorio	Preparó el commit; el push fue rechazado (sin permisos) y entregó un parche `git am`.
3	"Ya solo falta el III.4 y la bitácora de IA, ¿ya está hecho?"	Completar dos puntos	Respondió que no estaban hechos; redactó una bitácora y pidió el enunciado del III.4.
4	Se adjuntó el PDF del examen	Dar el enunciado	Leyó las 5 páginas, resolvió el III.4 con los datos de la versión SplitMix64 y señaló diferencias con los entregables (nombre `montecarlo.c`, uso de `rand_r`, protocolo de la tabla).
5	Pedido de desarrollar la evaluación completa (Partes I–IV, opcionales, pruebas, informe, bitácora, sustentación) con reglas de rigor y honestidad	Solución completa y verificable	Todo el contenido actual del repositorio (detalle en la sección 2). Reemplazó los resultados y el III.4 de las consultas 1 y 4.
6	"Sí, haz los 3 puntos"	Completar los puntos señalados en la consulta 4	Indicó que ya estaban resueltos en la consulta 5 (`montecarlo.c`, `rand_r` por hilo, tabla con el protocolo) y pidió los datos que solo el equipo puede dar.
7	El equipo indicó que no usó otras herramientas de IA, que pondrá los nombres y que la semilla asignada es 20261018	Cerrar pendientes	Actualizó esta bitácora y la lista de verificación; no cambió código ni resultados (la semilla ya era 20261018).

2. Propuestas y decisiones de la herramienta (consulta 5)

Tema	Propuesta / decisión de la herramienta	Alternativas descartadas por la herramienta y por qué	Evidencia generada	Evaluación del estudiante
Generador por hilo (II.2)	Estado `rand_r` privado por bloque, posicionado con un **salto del LCG** de glibc ($6i_0$ pasos); resultado idéntico para todo p	(a) Semilla $s + tN/p$: puede solaparse (se demostró con $p = 8$). (b) SplitMix64 del compañero: no usa `rand_r` ni la secuencia del docente; se conservó en `src/variantes/`. (c)	`tests/test_generador.c`; `resultados/solapamiento.txt`; prueba 7 de `run_tests.sh`	

Tema	Propuesta / decisión de la herramienta	Alternativas descartadas por la herramienta y por qué	Evidencia generada	Evaluación del estudiante
		`rand_r` compartido: carrera.		
Dependencia de glibc	Verificar al iniciar que `rand_r` es el LCG; si no, abortar	Reimplementar `rand_r` dentro del programa: portable, pero se alejaba del "usar `rand_r`" del enunciado	Función `rand_r_es_lcg_glibc`	
Reparto del ciclo	`parallel for` sobre bloques de 65 536 muestras con `reduction(+:dentro)`	Saltar en cada muestra (unas 30 multiplicaciones extra por muestra: lento); una región `parallel` con rangos manuales (no permitía comparar `static` y `dynamic`)	Tiempos en `resultados/`	
$T(1)$	La versión secuencial real; también se reporta OpenMP con $p = 1$	Usar solo OpenMP con $p = 1$ (infla el speedup un 2 %)	Tabla de la Parte III	
Protocolo de medición	Calentamiento, 7 repeticiones intercaladas con orden rotado, mediana; 11 repeticiones en II.5	5 repeticiones seguidas por configuración (más sensibles a perturbaciones pasajeras)	`scripts/benchmark.py`, CSV con cada ejecución	
II.3 / II.4	Experimento controlado `defectos.c` (estado global, misma semilla, sin reducción)	Explicar solo en teoría	`resultados/defectos.txt`	
II.5	Concluir "sin efecto medible con $p = 4$"; la ventaja de `dynamic` con $p = 8$ se presenta como hipótesis	Declarar una política "mejor"	`resultados/tabla_schedule.md`	
Informe y documentos	Redacción de `docs/`, `informe/informe.md` y `docs/sustentacion.md`	—	PDF generado por `scripts/generar_informe.py`	

3. Errores de la herramienta detectados y corregidos

Error	Cómo se detectó	Corrección
Error estándar de $\hat{\pi}$ mal calculado en un README previo	Revisión de la herramienta antes de publicar	$\sigma = 4\sqrt{q(1-q)/N}$, recalculado
Primer benchmark (SplitMix64, $N = 2 \cdot 10^8$) con $p = 8$ más rápido que $p = 4$ en 4 núcleos	Resultado incompatible con el hardware	Se repitió con más muestras y repeticiones; luego se rediseñó el protocolo
Gustafson calculado con f de Amdahl en lugar de α	Revisión de la fórmula al redactar III.5	$\alpha = t_s/T(4)$, $S_G(64) = 61,4$
Advertencia de `-Wconversion` (`long long` → `double`)	Compilación con todas las advertencias	Conversión explícita
Afirmación "3 a 18 veces más lento"	Verificación contra `resultados/defectos.txt`	"2 a 18 veces"
El III.4 de la consulta 4 usaba datos de la versión SplitMix64	Rediseño en la consulta 5	Rehecho con las mediciones de `montecarlo.c`

4. Verificaciones

Ejecutadas por la herramienta (con su salida registrada en el repositorio o reproducible con `make`): compilación sin advertencias, `make test` (45 comprobaciones, todas aprobadas), `tests/test_resultados.py`, las mediciones y los experimentos de `resultados/`.

A cargo del estudiante (no hechas todavía):

- ☐ Leer y entender `montecarlo.c` línea por línea, en especial `lcg_saltar` y la reducción.
- ☐ Ejecutar `make test` en su propio equipo.
- ☐ Decidir si la tabla final se mide en su equipo (`make equipo benchmark analisis informe`); el examen pide adjuntar el `lscpu` **del equipo en que se midió**.
- ☐ Completar nombres en el informe y la reflexión personal.
- ☐ Llenar la columna “Evaluación del estudiante”.

Ya confirmado por el equipo: la semilla asignada es 20261018 (la usada en todo el trabajo) y no se usaron otras herramientas de IA.

5. Evaluación personal del apoyo recibido (editable por el estudiante)

> *Escriba aquí, con sus palabras: qué partes entiende y puede defender, qué propuestas aceptó o modificó y por qué, qué verificó por su cuenta y qué haría distinto.*